

a single microinstruction. However, if the full capability of the wider microprogram word is not heavily used, then much of the control store will be wasted, and the machine could be made smaller and faster by restricting the microinstruction capability.

The narrow, but usually longer, approach is often called *vertical microcode*, while the wide but short approach is called *horizontal microcode*. It should be noted that the terms “vertical microcode” and “horizontal microcode” have no universal definition—the designers of the 8086 considered its 21-bit microinstruction to be more horizontal than in other single-chip computers of the time. The related terms *maximally encoded* and *minimally encoded* are probably better than vertical and horizontal.

C.6 Concluding Remarks

We began this appendix by looking at how to translate a finite-state diagram to an implementation using a finite-state machine. We then looked at explicit sequencers that use a different technique for realizing the next-state function. Although large microprograms are often targeted at implementations using this explicit next-state approach, we can also implement a microprogram with a finite-state machine. As we saw, both ROM and PLA implementations of the logic functions are possible. The advantages of explicit versus encoded next state and ROM versus PLA implementation are summarized below.

The BIG Picture

Independent of whether the control is represented as a finite-state diagram or as a microprogram, translation to a hardware control implementation is similar. Each state or microinstruction asserts a set of control outputs and specifies how to choose the next state.

The next-state function may be implemented by either encoding it in a finite-state machine or using an explicit sequencer. The explicit sequencer is more efficient if the number of states is large and there are many sequences of consecutive states without branching.

The control logic may be implemented with either ROMs or PLAs (or even a mix). PLAs are more efficient unless the control function is very dense. ROMs may be appropriate if the control is stored in a separate memory, as opposed to within the same chip as the datapath.

C.7 Exercises

C.1 [10] <§C.2> Instead of using four state bits to implement the finite-state machine in [Figure C.3.1](#), use nine state bits, each of which is a 1 only if the finite-state machine is in that particular state (e.g., S1 is 1 in state 1, S2 is 1 in state 2, etc.). Redraw the PLA ([Figure C.3.9](#)).

C.2 [5] <§C.3> We wish to add the instruction `jal` (jump and link). Make any necessary changes to the datapath or to the control signals if needed. You can photocopy figures to make it faster to show the additions. How many product terms are required in a PLA that implements the control for the single-cycle datapath for `jal`?

C.3 [5] <§C.3> Now we wish to add the instruction `addi` (add immediate). Add any necessary changes to the datapath and to the control signals. How many product terms are required in a PLA that implements the control for the single-cycle datapath for `addi`?

C.4 [10] <§C.3> Determine the number of product terms in a PLA that implements the finite-state machine for `addi`. The easiest way to do this is to construct the additions to the truth tables for `addi`.

C.5 [20] <§C.4> Implement the finite-state machine of using an explicit counter to determine the next state. Fill in the new entries for the additions to [Figure C.4.5](#). Also, add any entries needed to the dispatch ROMs of [Figure C.5.2](#).

C.6 [15] <§§C.3–C.6> Determine the size of the PLAs needed to implement the multicycle machine, assuming that the next-state function is implemented with a counter. Implement the dispatch tables of [Figure C.5.2](#) using two PLAs and the contents of the main control unit in [Figure C.4.5](#) using another PLA. How does the total size of this solution compare to the single PLA solution with the next state encoded? What if the main PLAs for both approaches are split into two separate PLAs by factoring out the next-state or address select signals?

This page intentionally left blank

D

A P P E N D I X

*RISC: any computer
announced after 1985.*

A Survey of RISC Architectures for Desktop, Server, and Embedded Computers

Steven Przybylskic
A Designer of the Stanford MIPS

D.1	Introduction	D-3
D.2	Addressing Modes and Instruction Formats	D-5
D.3	Instructions: the MIPS Core Subset	D-9
D.4	Instructions: Multimedia Extensions of the Desktop/Server RISCs	D-16
D.5	Instructions: Digital Signal-Processing Extensions of the Embedded RISCs	D-19
D.6	Instructions: Common Extensions to MIPS Core	D-20
D.7	Instructions Unique to MIPS-64	D-25
D.8	Instructions Unique to Alpha	D-27
D.9	Instructions Unique to SPARC v9	D-29
D.10	Instructions Unique to PowerPC	D-32
D.11	Instructions Unique to PA-RISC 2.0	D-34
D.12	Instructions Unique to ARM	D-36
D.13	Instructions Unique to Thumb	D-38
D.14	Instructions Unique to SuperH	D-39
D.15	Instructions Unique to M32R	D-40
D.16	Instructions Unique to MIPS-16	D-40
D.17	Concluding Remarks	D-43

D.1

Introduction

We cover two groups of *reduced instruction set computer* (RISC) architectures in this appendix. The first group is the desktop and server RISCs:

- Digital Alpha
- Hewlett-Packard PA-RISC
- IBM and Motorola PowerPC
- MIPS INC MIPS-64
- Sun Microsystems SPARC

The second group is the embedded RISCs:

- Advanced RISC Machines ARM
- Advanced RISC Machines Thumb
- Hitachi SuperH
- Mitsubishi M32R
- MIPS INC MIPS-16

	Alpha	MIPS I	PA-RISC 1.1	PowerPC	SPARCv8
Date announced	1992	1986	1986	1993	1987
Instruction size (bits)	32	32	32	32	32
Address space (size, model)	64 bits, flat	32 bits, flat	48 bits, segmented	32 bits, flat	32 bits, flat
Data alignment	Aligned	Aligned	Aligned	Unaligned	Aligned
Data addressing modes	1	1	5	4	2
Protection	Page	Page	Page	Page	Page
Minimum page size	8 KB	4 KB	4 KB	4 KB	8 KB
I/O	Memory mapped	Memory mapped	Memory mapped	Memory mapped	Memory mapped
Integer registers (number, model, size)	31 GPR $\times$ 64 bits	31 GPR $\times$ 32 bits	31 GPR $\times$ 32 bits	32 GPR $\times$ 32 bits	31 GPR $\times$ 32 bits
Separate floating-point registers	31 $\times$ 32 or 31 $\times$ 64 bits	16 $\times$ 32 or 16 $\times$ 64 bits	56 $\times$ 32 or 28 $\times$ 64 bits	32 $\times$ 32 or 32 $\times$ 64 bits	32 $\times$ 32 or 32 $\times$ 64 bits
Floating-point format	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double	IEEE 754 single, double

FIGURE D.1.1 Summary of the first version of five architectures for desktops and servers. Except for the number of data address modes and some instruction set details, the integer instruction sets of these architectures are very similar. Contrast this with [Figure D.17.1](#). Later versions of these architectures all support a flat, 64-bit address space.

	ARM	Thumb	SuperH	M32R	MIPS-16
Date announced	1985	1995	1992	1997	1996
Instruction size (bits)	32	16	16	16/32	16/32
Address space (size, model)	32 bits, flat	32 bits, flat	32 bits, flat	32 bits, flat	32/64 bits, flat
Data alignment	Aligned	Aligned	Aligned	Aligned	Aligned
Data addressing modes	6	6	4	3	2
Integer registers (number, model, size)	15 GPR $\times$ 32 bits	8 GPR + SP, LR $\times$ 32 bits	16 GPR $\times$ 32 bits	16 GPR $\times$ 32 bits	8 GPR + SP, RA $\times$ 32/64 bits
I/O	Memory mapped	Memory mapped	Memory mapped	Memory mapped	Memory mapped

FIGURE D.1.2 Summary of five architectures for embedded applications. Except for number of data address modes and some instruction set details, the integer instruction sets of these architectures are similar. Contrast this with [Figure D.17.1](#).

There has never been another class of computers so similar. This similarity allows the presentation of 10 architectures in about 50 pages. Characteristics of the desktop and server RISCs are found in [Figure D.1.1](#) and the embedded RISCs in [Figure D.1.2](#).

Notice that the embedded RISCs tend to have eight to 16 general-purpose registers while the desktop/server RISCs have 32, and that the length of instructions is 16 to 32 bits in embedded RISCs but always 32 bits in desktop/server RISCs.

Although shown as separate embedded instruction set architectures, Thumb and MIPS-16 are really optional modes of ARM and MIPS invoked by call instructions. When in this mode, they execute a subset of the native architecture using 16-bit-long instructions. These 16-bit instruction sets are not intended to be full architectures, but they are enough to encode most procedures. Both machines expect procedures to be homogeneous, with all instructions in either 16-bit mode or 32-bit mode. Programs will consist of procedures in 16-bit mode for density or in 32-bit mode for performance.

One complication of this description is that some of the older RISCs have been extended over the years. We have decided to describe the latest versions of the architectures: MIPS-64, Alpha version 3, PA-RISC 2.0, and SPARC version 9 for the desktop/server; ARM version 4, Thumb version 1, Hitachi SuperH SH-3, M32R version 1, and MIPS-16 version 1 for the embedded ones.

The remaining sections proceed as follows: after discussing the addressing modes and instruction formats of our RISC architectures, we present the survey of the instructions in five steps:

- Instructions found in the MIPS core, which is defined in Chapters 2 and 3 of the main text
- Multimedia extensions of the desktop/server RISCs
- Digital signal-processing extensions of the embedded RISCs
- Instructions not found in the MIPS core but found in two or more architectures
- The unique instructions and characteristics of each of the 10 architectures

We give the evolution of the instruction sets in the final section and conclude with speculation about future directions for RISCs.

D.2

Addressing Modes and Instruction Formats

[Figure D.2.1](#) shows the data addressing modes supported by the desktop architectures. Since all have one register that always has the value 0 when used in address modes, the absolute address mode with limited range can be synthesized using zero as the base in displacement addressing. (This register can be changed

by ALU operations in PowerPC; it is always 0 in the other machines.) Similarly, register indirect addressing is synthesized by using displacement addressing with an offset of 0. Simplified addressing modes is one distinguishing feature of RISC architectures.

Figure D.2.2 shows the data addressing modes supported by the embedded architectures. Unlike the desktop RISCs, these embedded machines do not reserve a register to contain 0. Although most have two to three simple addressing modes, ARM and SuperH have several, including fairly complex calculations. ARM has an addressing mode that can shift one register by any amount, add it to the other registers to form the address, and then update one register with this new address.

References to code are normally PC-relative, although jump register indirect is supported for returning from procedures, for *case* statements, and for pointer function calls. One variation is that PC-relative branch addresses are shifted left 2 bits before being added to the PC for the desktop RISCs, thereby increasing the branch distance. This works because the length of all instructions for the desktop RISCs is 32 bits, and instructions must be aligned on 32-bit words in memory. Embedded architectures with 16-bit-long instructions usually shift the PC-relative address by one for similar reasons.

Addressing mode	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Register + offset (displacement or based)	X	X	X	X	X
Register + register (indexed)		X (FP)	X (Loads)	X	X
Register + scaled register (scaled)			X		
Register + offset and update register			X	X	
Register + register and update register			X	X	

FIGURE D.2.1 Summary of data addressing modes supported by the desktop architectures. PA-RISC also has short address versions of the offset addressing modes. MIPS-64 has indexed addressing for floating-point loads and stores. (These addressing modes are described in Figure 2.18.)

Addressing mode	ARMv4	Thumb	SuperH	M32R	MIPS-16
Register + offset (displacement or based)	X	X	X	X	X
Register + register (indexed)	X	X	X		
Register + scaled register (scaled)	X				
Register + offset and update register	X				
Register + register and update register	X				
Register indirect			X	X	
Autoincrement, autodecrement	X	X	X	X	
PC-relative data	X	X (loads)	X		X (loads)

FIGURE D.2.2 Summary of data addressing modes supported by the embedded architectures. SuperH and M32R have separate register indirect and register + offset addressing modes rather than just putting 0 in the offset of the latter mode. This increases the use of 16-bit instructions in the M32R, and it gives a wider set of address modes to different data transfer instructions in SuperH. To get greater addressing range, ARM and Thumb shift the offset left one or two bits if the data size is halfword or word. (These addressing modes are described in Figure 2.18.)

Figure D.2.3 shows the format of the desktop RISC instructions, which include the size of the address. Each instruction set architecture uses these four primary instruction formats. Figure D.2.4 shows the six formats for the embedded RISC machines. The desire to have smaller code size via 16-bit instructions leads to more instruction formats.

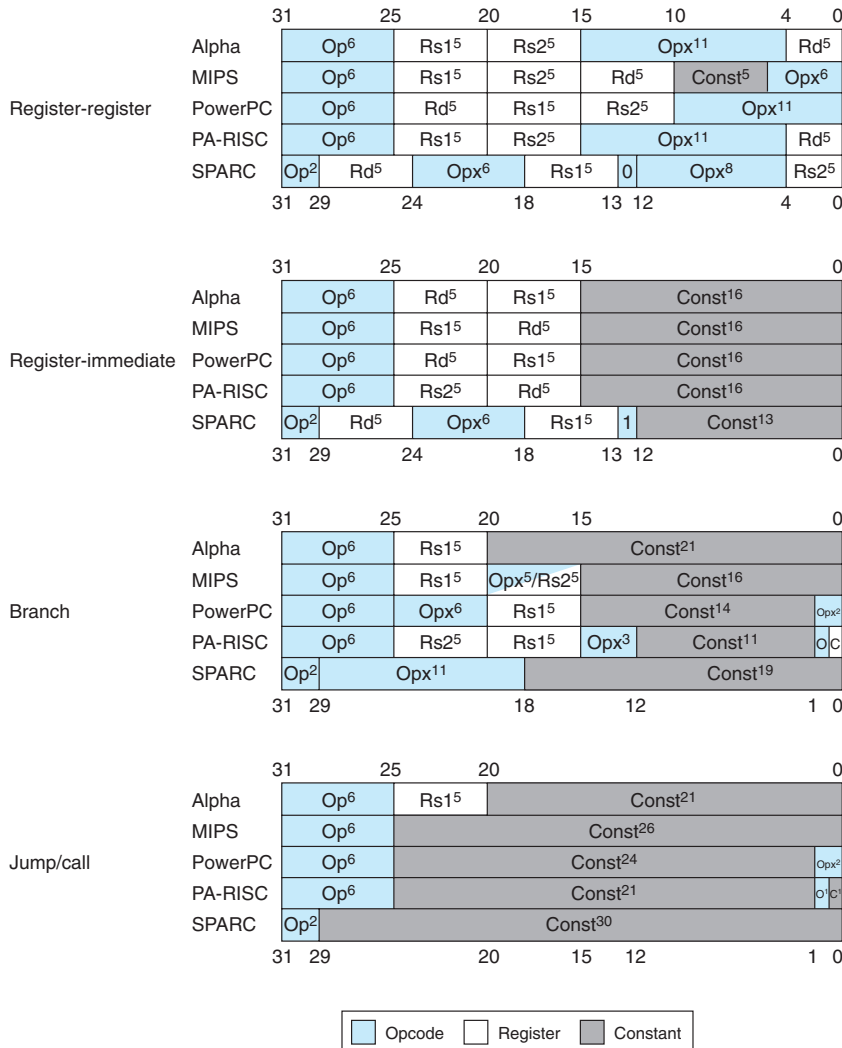


FIGURE D.2.3 Instruction formats for desktop/server RISC architectures. These four formats are found in all five architectures. (The superscript notation in this figure means the width of a field in bits.) Although the register fields are located in similar pieces of the instruction, be aware that the destination and two source fields are scrambled. Op = the main opcode, Opx = an opcode extension, Rd = the destination register, Rs1 = source register 1, Rs2 = source register 2, and Const = a constant (used as an immediate or as an address). Unlike the other RISCs, Alpha has a format for immediates in arithmetic and logical operations that is different from the data transfer format shown here. It provides an 8-bit immediate in bits 20 to 13 of the RR format, with bits 12 to 5 remaining as an opcode extension.

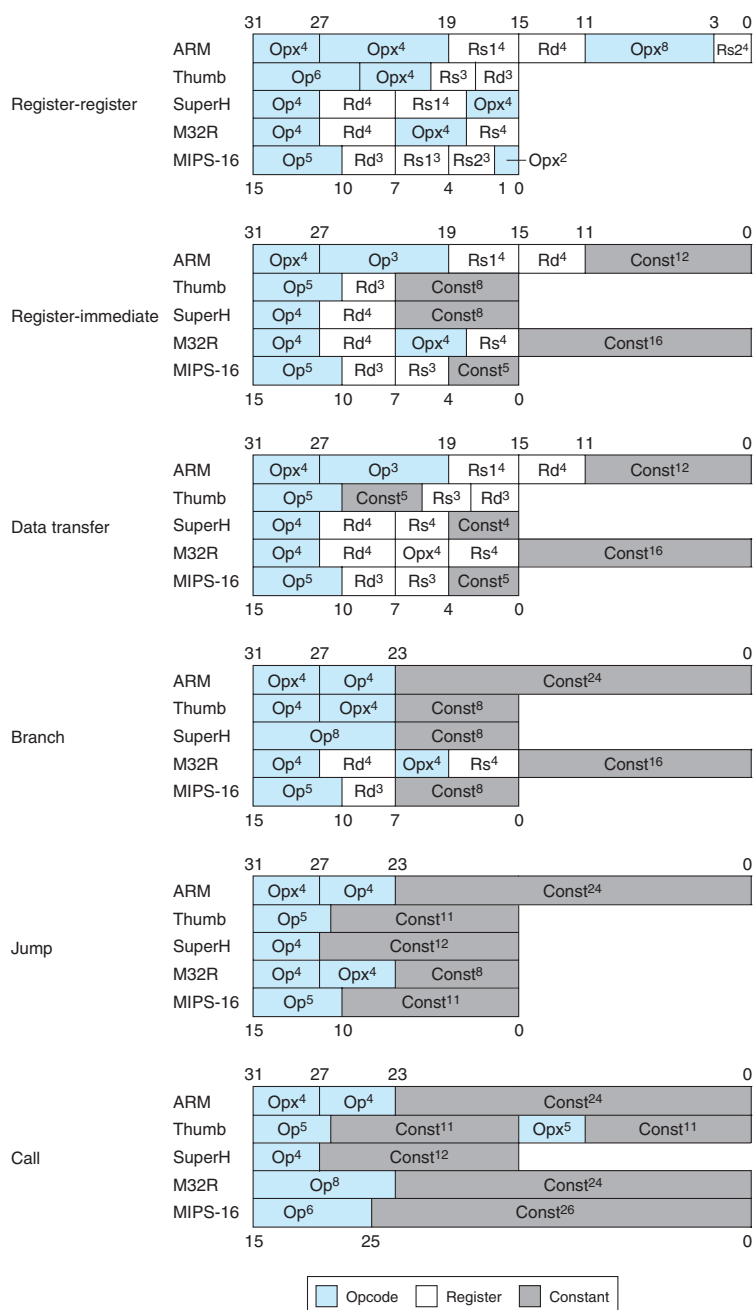


FIGURE D.2.4 Instruction formats for embedded RISC architectures. These six formats are found in all five architectures. The notation is the same as in Figure D.2.3. Note the similarities in branch, jump, and call formats, and the diversity in register-register, register-immediate, and data transfer formats. The differences result from whether the architecture has eight or 16 registers, whether it is a two- or three-operand format, and whether the instruction length is 16 or 32 bits.

Format: instruction category	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Branch: all	Sign	Sign	Sign	Sign	Sign
Jump/call: all	Sign	—	Sign	Sign	Sign
Register-immediate: data transfer	Sign	Sign	Sign	Sign	Sign
Register-immediate: arithmetic	Zero	Sign	Sign	Sign	Sign
Register-immediate: logical	Zero	Zero	—	Zero	Sign

FIGURE D.2.5 Summary of constant extension for desktop RISCs. The constants in the jump and call instructions of MIPS are not sign-extended, since they only replace the lower 28 bits of PC, leaving the upper 4 bits unchanged. PA-RISC has no logical immediate instructions.

Format: instruction category	Armv4	Thumb	SuperH	M32R	MIPS-16
Branch: all	Sign	Sign	Sign	Sign	Sign
Jump/call: all	Sign	Sign/Zero	Sign	Sign	—
Register-immediate: data transfer	Zero	Zero	Zero	Sign	Zero
Register-immediate: arithmetic	Zero	Zero	Sign	Sign	Zero/Sign
Register-immediate: logical	Zero	—	Zero	Zero	—

FIGURE D.2.6 Summary of constant extension for embedded RISCs. The 16-bit-length instructions have much shorter immediates than those of the desktop RISCs, typically only 5 to 8 bits. Most embedded RISCs, however, have a way to get a long address for procedure calls from two sequential halfwords. The constants in the jump and call instructions of MIPS are not sign-extended, since they only replace the lower 28 bits of the PC, leaving the upper 4 bits unchanged. The 8-bit immediates in ARM can be rotated right an even number of bits between 2 and 30, yielding a large range of immediate values. For example, all powers of two are immediates in ARM.

Figures D.2.5 and D.2.6 show the variations in extending constant fields to the full width of the registers. In this subtle point, the RISCs are similar but not identical.

D.3 Instructions: the MIPS Core Subset

The similarities of each architecture allow simultaneous descriptions, starting with the operations equivalent to the MIPS core.

MIPS Core Instructions

Almost every instruction found in the MIPS core is found in the other architectures, as Figures D.3.1 through D.3.5 show. (For reference, definitions of the MIPS instructions are found in the MIPS Reference Data Card at the beginning of the book.) Instructions are listed under four categories: data transfer (Figure D.3.1); arithmetic/logical (Figure D.3.2); control (Figure D.3.3); and floating point (Figure D.3.4). A fifth category (Figure D.3.5) shows conventions for register

Data transfer (instruction formats)	R-I	R-I	R-I, R-R	R-I, R-R	R-I, R-R
Instruction name	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Load byte signed	LDBU; SEXTB	LB	LDB; EXTRW, S 31, 8	LBZ; EXTSB	LDSB
Load byte unsigned	LDBU	LBU	LDB, LDBX, LDBS	LBZ	LDUB
Load halfword signed	LDWU; SEXTW	LH	LDH; EXTRW, S 31, 16	LHA	LDSH
Load halfword unsigned	LDWU	LHU	LDH, LDHX, LDHS	LHZ	LDUH
Load word	LDLS	LW	LDW, LDWX, LDWS	LW	LD
Load SP float	LDS*	LWC1	FLDWX, FLDWS	LFS	LDF
Load DP float	LDT	LDC1	FLDDX, FLDDS	LFD	LDDF
Store byte	STB	SB	STB, STBX, STBS	STB	STB
Store halfword	STW	SH	STH, STHX, STHS	STH	STH
Store word	STL	SW	STW, STWX, STWS	STW	ST
Store SP float	STS	SWC1	FSTWX, FSTWS	STFS	STF
Store DP float	STT	SDC1	FSTDY, FSTDY	STFD	STDF
Read, write special registers	MF_, MT_	MF, MT_	MFCTL, MTCTL	MFSPR, MF_, MTSPR, MT_	RD, WR, RDPR, WRPR, LDXFSR, STXFSR
Move integer to FP register	ITOFs	MFC1/DMFC1	STW; FLDWX	STW; LDFS	ST; LDF
Move FP to integer register	FTTOIS	MTC1/DMTC1	FSTWX; LDW	STFS; LW	STF; LD

FIGURE D.3.1 Desktop RISC data transfer instructions equivalent to MIPS core. A sequence of instructions to synthesize a MIPS instruction is shown separated by semicolons. If there are several choices of instructions equivalent to MIPS core, they are separated by commas. For this figure, halfword is 16 bits and word is 32 bits. Note that in Alpha, LDS converts single-precision floating point to double precision and loads the entire 64-bit register.

usage and pseudoinstructions on each architecture. If a MIPS core instruction requires a short sequence of instructions in other architectures, these instructions are separated by semicolons in [Figures D.3.1 through D.3.5](#). (To avoid confusion, the destination register will always be the leftmost operand in this appendix, independent of the notation normally used with each architecture.) [Figures D.3.6 through D.3.9](#) show the equivalent listing for embedded RISCs. Note that floating point is generally not defined for the embedded RISCs.

Every architecture must have a scheme for compare and conditional branch, but despite all the similarities, each of these architectures has found a different way to perform the operation.

Compare and Conditional Branch

SPARC uses the traditional four condition code bits stored in the program status word: *negative*, *zero*, *carry*, and *overflow*. They can be set on any arithmetic or logical instruction; unlike earlier architectures, this setting is optional on each instruction. An explicit option leads to fewer problems in pipelined implementation. Although condition codes can be set as a side effect of an operation, explicit compares are synthesized with a subtract using r0 as the destination. SPARC conditional branches

Arithmetic/logical (instruction formats)	R-R, R-I	R-R, R-I	R-R, R-I	R-R, R-I	R-R, R-I
Instruction name	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Add	ADDL	ADDU, ADDU	ADDL, LDO, ADDI, UADDCM	ADD, ADDI	ADD
Add (trap if overflow)	ADDLV	ADD, ADDI	ADD0, ADDI0	ADD0; MCRXR; BC	ADDcc; TVS
Sub	SUBL	SUBU	SUB, SUBI	SUBF	SUB
Sub (trap if overflow)	SUBLV	SUB	SUBT0, SUBI0	SUBF/oe	SUBcc; TVS
Multiply	MULL	MULT, MULTU	SHiADD;...; (i=1,2,3)	MULLW, MULLI	MULX
Multiply (trap if overflow)	MULLV	—	SHiADD0;...;	—	—
Divide	—	DIV, DIVU	DS;...; DS	DIVW	DIVX
Divide (trap if overflow)	—	—	—	—	—
And	AND	AND, ANDI	AND	AND, ANDI	AND
Or	BIS	OR, ORI	OR	OR, ORI	OR
Xor	XOR	XOR, XORI	XOR	XOR, XORI	XOR
Load high part register	LDAH	LUI	LDIL	ADDIS	SETHI (B fmt.)
Shift left logical	SLL	SLLV, SLL	DEPW, Z 31-i, 32-i	RLWINM	SLL
Shift right logical	SRL	SRLV, SRL	EXTRW, U 31, 32-i	RLWINM 32-i	SRL
Shift right arithmetic	SRA	SRAV, SRA	EXTRW, S 31, 32-i	SRAW	SRA
Compare	CMPEQ, CMPLT, CMPL	SLT/U, SLTI/U	COMB	CMP(I)CLR	SUBcc r0,...

FIGURE D.3.2 Desktop RISC arithmetic/logical instructions equivalent to MIPS core. Dashes mean the operation is not available in that architecture, or not synthesized in a few instructions. Such a sequence of instructions is shown separated by semicolons. If there are several choices of instructions equivalent to MIPS core, they are separated by commas. Note that in the “Arithmetic/logical” category, all machines but SPARC use separate instruction mnemonics to indicate an immediate operand; SPARC offers immediate versions of these instructions but uses a single mnemonic. (Of course these are separate opcodes!)

Control (instruction formats)	B, J/C	B, J/C	B, J/C	B, J/C	B, J/C
Instruction name	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Branch on integer compare	B_(<, >, <=, >=, =, not=)	BEQ, BNE, B_Z(<, >, <=, >=)	COMB, COMIB	BC	BR_Z, BPcc(<, >, <=, >=, =, not=)
Branch on floating-point compare	FB_(<, >, <=, >=, =, not=)	BC1T, BC1F	FSTWX f0; LDW t; BB t	BC	FBPfcc(<, >, <=, >=, =, ...)
Jump, jump register	BR, JMP	J, JR	BL r0, BLR r0	B, BCLR, BCCTR	BA, JMPL r0,...
Call, call register	BSR	JAL, JALR	BL, BLE	BL, BLA, BCLRL, BCCTRL	CALL, JMPL
Trap	CALL_PAL GENTRAP	BREAK	BREAK	TW, TWI	Ticc, SIR
Return from interrupt	CALL_PAL REI	JR; ERET	RFI, RFIR	RFI	DONE, RETRY, RETURN

FIGURE D.3.3 Desktop RISC control instructions equivalent to MIPS core. If there are several choices of instructions equivalent to MIPS core, they are separated by commas.

Floating point (instruction formats)	R-R	R-R	R-R	R-R	R-R
Instruction name	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Add single, double	ADDS, ADDT	ADD.S, ADD.D	FADD FADD/db1	FADDS, FADD	FADDS, FADD
Subtract single, double	SUBS, SUBT	SUB.S, SUB.D	FSUB FSUB/db1	FSUBS, FSUB	FSUBS, FSUBD
Multiply single, double	MULS, MULT	MUL.S, MUL.D	FMPY FMPY/db1	FMULS, FMUL	FMULS, FMULD
Divide single, double	DIVS, DIVT	DIV.S, DIV.D	FDIV, FDIV/db1	FDIVS, FDIV	FDIVS, FDIVD
Compare	CMPT_ (=, <, <=, UN)	C_.S, C_.D (<, >, <=, >=, =, ...)	FCMP, FCMP/db1 (<, =, >)	FCMP	FCMPS, FCMPD
Move R-R	ADDT Fd, F31, Fs	MOV.S, MOV.D	FCPY	FMV	FMOVS/D/Q
Convert (single, double, integer) to (single, double, integer)	CVTST, CVTTS, CVTTQ, CVTQS, CVTQT	CVT.S.D, CVT.D.S, CVT.S.W, CVT.D.W, CVT.W.S, CVT.W.D	FCNVFF,s,d FCNVFF,d,s FCNVXF,s,s FCNVXF,d,d FCNVFX,s,s FCNVFX,d,s	—, FRSP, —, FCTIW,—, —	FSTOD, FDTOS, FSTOI, FDTOI, FITOS, FITOD

FIGURE D.3.4 Desktop RISC floating-point instructions equivalent to MIPS core. Dashes mean the operation is not available in that architecture, or not synthesized in a few instructions. If there are several choices of instructions equivalent to MIPS core, they are separated by commas.

Conventions	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Register with value 0	r31 (source)	r0	r0	r0 (addressing)	r0
Return address register	(any)	r31	r2, r31	link (special)	r31
No-op	LDQ_U r31, ...	SLL r0, r0, r0	OR r0, r0, r0	ORI r0, r0, #0	SETHI r0, 0
Move R-R integer	BIS..., r31, ...	ADD..., r0, ...	OR..., r0, ...	OR rx, ry, ry	OR..., r0, ...
Operand order	OP Rs1, Rs2, Rd	OP Rd, Rs1, Rs2	OP Rs1, Rs2, Rd	OP Rd, Rs1, Rs2	OP Rs1, Rs2, Rd

FIGURE D.3.5 Conventions of desktop RISC architectures equivalent to MIPS core.

test condition codes to determine all possible unsigned and signed relations. Floating point uses separate condition codes to encode the IEEE 754 conditions, requiring a floating-point compare instruction. Version 9 expanded SPARC branches in four ways: a separate set of condition codes for 64-bit operations; a branch that tests the contents of a register and branches if the value is =, not =, <, <=, >=, or <= 0 (see MIPS below); three more sets of floating-point condition codes; and branch instructions that encode static branch prediction.

PowerPC also uses four condition codes—*less than*, *greater than*, *equal*, and *summary overflow*—but it has eight copies of them. This redundancy allows the PowerPC instructions to use different condition codes without conflict, essentially giving PowerPC eight extra 4-bit registers. Any of these eight condition codes can be the target of a compare instruction, and any can be the source of a conditional branch. The integer instructions have an option bit that behaves as if the integer op

Instruction name	ARMv4	Thumb	SuperH	M32R	MIPS-16
Data transfer (instruction formats)	DT	DT	DT	DT	DT
Load byte signed	LDRSB	LDRSB	MOV.B	LDB	LB
Load byte unsigned	LDRB	LDRB	MOV.B; EXTU.B	LDUB	LBU
Load halfword signed	LDRSH	LDRSH	MOV.W	LDH	LH
Load halfword unsigned	LDRH	LDRH	MOV.W; EXTU.W	LDUH	LHU
Load word	LDR	LDR	MOV.L	LD	LW
Store byte	STRB	STRB	MOV.B	STB	SB
Store halfword	STRH	STRH	MOV.W	STH	SH
Store word	STR	STR	MOV.L	ST	SW
Read, write special registers	MRS, MSR	— ¹	LDC, STC	MVFC, MVTC	MOVE

FIGURE D.3.6 Embedded RISC data transfer instructions equivalent to MIPS core. A sequence of instructions to synthesize a MIPS instruction is shown separated by semicolons. Note that floating point is generally not defined for the embedded RISCs. Thumb and MIPS-16 are just 16-bit instruction subsets of the ARM and MIPS architectures, so machines can switch modes and execute the full instruction set. We use —¹ to show sequences that are available in 32-bit mode but not 16-bit mode in Thumb or MIPS-16.

is followed by a compare to zero that sets the first condition “register.” PowerPC also lets the second “register” be optionally set by floating-point instructions. PowerPC provides logical operations among these eight 4-bit condition code registers (CRAND, CROR, CRXOR, CRNAND, CRNOR, CREQV), allowing more complex conditions to be tested by a single branch.

MIPS uses the contents of registers to evaluate conditional branches. Any two registers can be compared for equality (BEQ) or inequality (BNE), and then the branch is taken if the condition holds. The set on less than instructions (SLT, SLTI, SLTU, SLTIU) compare two operands and then set the destination register to 1 if less and to 0 otherwise. These instructions are enough to synthesize the full set of relations. Because of the popularity of comparisons to 0, MIPS includes special compare and branch instructions for all such comparisons: greater than or equal to zero (BGEZ), greater than zero (BGTZ), less than or equal to zero (BLEZ), and less than zero (BLTZ). Of course, equal and not equal to zero can be synthesized using r0 with BEQ and BNE. Like SPARC, MIPS I uses a condition code for floating point with separate floating-point compare and branch instructions; MIPS IV expanded this to eight floating-point condition codes, with the floating point comparisons and branch instructions specifying the condition to set or test.

Alpha compares (CMPEQ, CMPLT, CMPLE, CMPULT, CMPULE) test two registers and set a third to 1 if the condition is true and to 0 otherwise. Floating-point compares (CMTEQ, CMTLT, CMTLE, CMTUN) set the result to 2.0 if the condition holds and to 0 otherwise. The branch instructions compare one register to 0 (BEQ, BGE, BGT, BLE, BLT, BNE) or its least significant bit to 0 (BLBC, BLBS) and then branch if the condition holds.

Arithmetic/logical (instruction formats)	R-R, R-I	R-R, R-I	R-R, R-I	R-R, R-I	R-R, R-I
Instruction name	ARMv4	Thumb	SuperH	M32R	MIPS-16
Add	ADD	ADD	ADD	ADD, ADDI, ADD3	ADDU, ADDIU
Add (trap if overflow)	ADDS; SWIVS	ADD; BVC .+4; SWI	ADDV	ADDV, ADDV3	— ¹
Subtract	SUB	SUB	SUB	SUB	SUBU
Subtract (trap if overflow)	SUBS; SWIVS	SUB; BVC .+1; SWI	SUBV	SUBV	— ¹
Multiply	MUL	MUL	MUL	MUL	MULT, MULTU
Multiply (trap if overflow)					—
Divide	—	—	DIV1, DIVoS, DIVoU	DIV, DIVU	DIV, DIVU
Divide (trap if overflow)	—	—			—
And	AND	AND	AND	AND, AND3	AND
Or	ORR	ORR	OR	OR, OR3	OR
Xor	EOR	EOR	XOR	XOR, XOR3	XOR
Load high part register	—	—		SETH	— ¹
Shift left logical	LSL ³	LSL ²	SHLL, SHLLn	SLL, SLLI, SLL3	SLLV, SLL
Shift right logical	LSR ³	LSR ²	SHRL, SHRLn	SRL, SRLI, SRL3	SRLV, SRL
Shift right arithmetic	ASR ³	ASR ²	SHRA, SHAD	SRA, SRAI, SRA3	SRAV, SRA
Compare	CMP, CMN, TST, TEQ	CMP, CMN, TST	CMP/cond, TST	CMP/I, CMPI/I	CMP/I ² , SLT/I, SLT/IU

FIGURE D.3.7 Embedded RISC arithmetic/logical instructions equivalent to MIPS core. Dashes mean the operation is not available in that architecture, or not synthesized in a few instructions. Such a sequence of instructions is shown separated by semicolons. If there are several choices of instructions equivalent to MIPS core, they are separated by commas. Thumb and MIPS-16 are just 16-bit instruction subsets of the ARM and MIPS architectures, so machines can switch modes and execute the full instruction set. We use —¹ to show sequences that are available in 32-bit mode but not 16-bit mode in Thumb or MIPS-16. The superscript 2 shows new instructions found only in 16-bit mode of Thumb or MIPS-16, such as CMP/I². ARM includes shifts as part of every data operation instruction, so the shifts with superscript 3 are just a variation of a move instruction, such as LSR³.

PA-RISC has many branch options, which we'll see in Section D.11. The most straightforward is a compare and branch instruction (COMB), which compares two registers, branches depending on the standard relations, and then tests the least significant bit of the result of the comparison.

ARM is similar to SPARC, in that it provides four traditional condition codes that are optionally set. CMP subtracts one operand from the other and the difference sets the condition codes. Compare negative (CMN) adds one operand to the other, and the sum sets the condition codes. TST performs logical AND on the two operands to set all condition codes but overflow, while TEQ uses exclusive OR to set the first three condition codes. Like SPARC, the conditional version of the ARM branch instruction tests condition codes to determine all possible unsigned and signed relations.

Control (instruction formats)	B, J, C	B, J, C	B, J, C	B, J, C	B, J, C
Instruction name	ARMv4	Thumb	SuperH	M32R	MIPS-16
Branch on integer compare	B/cond	B/cond	BF, BT	BEQ, BNE, BC, BNC, B__Z	BEQZ ² , BNEZ ² , BTEQZ ² , BTNEZ ²
Jump, jump register	MOV pc, ri	MOV pc, ri	BRA, JMP	BRA, JMP	B ² , JR
Call, call register	BL	BL	BSR, JSR	BL, JL	JAL, JALR, JALX ²
Trap	SWI	SWI	TRAPA	TRAP	BREAK
Return from interrupt	MOVS pc, r14	__ ¹	RTS	RTE	__ ¹

FIGURE D.3.8 Embedded RISC control instructions equivalent to MIPS core. Thumb and MIPS-16 are just 16-bit instruction subsets of the ARM and MIPS architectures, so machines can switch modes and execute the full instruction set. We use __¹ to show sequences that are available in 32-bit mode but not 16-bit mode in Thumb or MIPS-16. The superscript 2 shows new instructions found only in 16-bit mode of Thumb or MIPS-16, such as BTEQZ².

Conventions	ARMv4	Thumb	SuperH	M32R	MIPS-16
Return address reg.	R14	R14	PR (special)	R14	RA (special)
No-op	MOV r0, r0	MOV r0, r0	NOP	NOP	SLL r0, r0
Operands, order	OP Rd, Rs1, Rs2	OP Rd, Rs1	OP Rs1, Rd	OP Rd, Rs1	OP Rd, Rs1, Rs2

FIGURE D.3.9 Conventions of embedded RISC instructions equivalent to MIPS core.

As we shall see in Section D.12, one unusual feature of ARM is that every instruction has the option of executing conditionally depending on the condition codes. (This bears similarities to the annulling option of PA-RISC, seen in Section D.11.)

Not surprisingly, Thumb follows ARM. The differences are that setting condition codes are not optional, the TEQ instruction is dropped, and there is no conditional execution of instructions.

The Hitachi SuperH uses a single T-bit condition that is set by compare instructions. Two branch instructions decide to branch if either the T bit is 1 (BT) or the T bit is 0 (BF). The two flavors of branches allow fewer comparison instructions.

Mitsubishi M32R also offers a single condition code bit (C) used for signed and unsigned comparisons (CMP, CMPI, CMPU, CMPUI) to see if one register is less than the other or not, similar to the MIPS set on less than instructions. Two branch instructions test to see if the C bit is 1 or 0: BC and BNC. The M32R also includes instructions to branch on equality or inequality of registers (BEQ and BNE) and all relations of a register to 0 (BGEZ, BGTZ, BLEZ, BLTZ, BEQZ, BNEZ). Unlike BC and BNC, these last instructions are all 32 bits wide.

MIPS-16 keeps set on less than instructions (SLT, SLTI, SLTU, SLTIU), but instead of putting the result in one of the eight registers, it is placed in a special register named T. MIPS-16 is always implemented in machines that also have the full 32-bit MIPS instructions and registers; hence, register T is really register 24 in the full MIPS architecture. The MIPS-16 branch instructions test to see if a register is or is not equal to zero (BEQZ and BNEZ). There are also instructions that branch

	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Number of condition code bits (integer and FP)	0	8 FP	8 FP	8 × 4 both	2 × 4 integer, 4 × 2 FP
Basic compare instructions (integer and FP)	1 integer, 1 FP	1 integer, 1 FP	4 integer, 2 FP	4 integer, 2 FP	1 FP
Basic branch instructions (integer and FP)	1	2 integer, 1 FP	7 integer	1 both	3 integer, 1 FP
Compare register with register/const and branch	—	=, not=	=, not=, <, <=, >, >=, even, odd	—	—
Compare register to zero and branch	=, not=, <, <=, >, >=, even, odd	=, not=, <, <=, >, >=	=, not=, <, <=, >, >=, even, odd	—	=, not=, <, <=, >, >=

FIGURE D.3.10 Summary of five desktop RISC approaches to conditional branches. Floating-point branch on PA-RISC is accomplished by copying the FP status register into an integer register and then using the branch on bit instruction to test the FP comparison bit. Integer compare on SPARC is synthesized with an arithmetic instruction that sets the condition codes using r0 as the destination.

	ARMv4	Thumb	SuperH	M32R	MIPS-16
Number of condition code bits	4	4	1	1	1
Basic compare instructions	4	3	2	2	2
Basic branch instructions	1	1	2	3	2
Compare register with register/const and branch	—	—	=, >, >=	=, not=	—
Compare register to zero and branch	—	—	=, >, >=	=, not=, <, <=, >, >=	=, not=

FIGURE D.3.11 Summary of five embedded RISC approaches to conditional branches.

if register T is or is not equal to zero (BTEQZ and BTNEZ). To test if two registers are equal, MIPS added compare instructions (CMP , CMPI) that compute the exclusive OR of two registers and place the result in register T. Compare was added since MIPS-16 left out instructions to compare and branch if registers are equal or not (BEQ and BNE).

Figures D.3.10 and D.3.11 summarize the schemes used for conditional branches.

D.4

Instructions: Multimedia Extensions of the Desktop/Server RISCs

Since every desktop microprocessor by definition has its own graphical displays, as transistor budgets increased it was inevitable that support would be added for graphics operations. Many graphics systems use 8 bits to represent each of the three primary colors plus 8 bits for the location of a pixel.

The addition of speakers and microphones for teleconferencing and video games suggested support of sound as well. Audio samples need more than 8 bits of precision, but 16 bits are sufficient.

Every microprocessor has special support so that bytes and halfwords take up less space when stored in memory, but due to the infrequency of arithmetic operations on these data sizes in typical integer programs, there is little support beyond data transfers. The architects of the Intel i860, which was justified as a graphical accelerator within the company, recognized that many graphics and audio applications would perform the same operation on vectors of these data. Although a vector unit was beyond the transistor budget of the i860 in 1989, by partitioning the carry chains within a 64-bit ALU, it could perform simultaneous operations on short vectors of eight 8-bit operands, four 16-bit operands, or two 32-bit operands. The cost of such partitioned ALUs was small. Applications that lend themselves to such support include MPEG (video), games like DOOM (3-D graphics), Adobe Photoshop (digital photography), and teleconferencing (audio and image processing).

Like a virus, over time such multimedia support has spread to nearly every desktop microprocessor. HP was the first successful desktop RISC to include such support. As we shall see, this virus spread unevenly. The PowerPC is the only holdout, and rumors are that it is “running a fever.”

These extensions have been called subword parallelism, vector, or SIMD (single-instruction, multiple data) (see [Chapter 6](#)). Since Intel marketing uses SIMD to describe the MMX extension of the 8086, that has become the popular name. [Figure D.4.1](#) summarizes the support by architecture.

From [Figure D.4.1](#), you can see that, in general, MIPS MDMX works on eight bytes or four halfwords per instruction, HP PA-RISC MAX2 works on four halfwords, SPARC VIS works on four halfwords or two words, and Alpha doesn't do much. The Alpha MAX operations are just byte versions of compare, min, max, and absolute difference, leaving it up to software to isolate fields and perform parallel adds, subtracts, and multiplies on bytes and halfwords. MIPS also added operations to work on two 32-bit floating-point operands per cycle, but they are considered part of MIPS V and not simply multimedia extensions (see [Section D.7](#)).

One feature not generally found in general-purpose microprocessors is saturating operations. Saturation means that when a calculation overflows, the result is set to the largest positive number or most negative number, rather than a modulo calculation as in two's complement arithmetic. Commonly found in digital signal processors (see the next section), these saturating operations are helpful in routines for filtering.

These machines largely used existing register sets to hold operands: integer registers for Alpha and HP PA-RISC and floating-point registers for MIPS and Sun. Hence data transfers are accomplished with standard load and store instructions. MIPS also added a 192-bit (3×64) wide register to act as an accumulator for some operations. By having three times the native data width, it can be partitioned to accumulate either eight bytes with 24 bits per field or four halfwords with 48 bits

Instruction category	Alpha MAX	MIPS MDMX	PA-RISC MAX2	PowerPC	SPARC VIS
Add/subtract		8B, 4H	4H		4H, 2W
Saturating add/sub		8B, 4H	4H		
Multiply		8B, 4H			4B/H
Compare	8B ($\geq$)	8B, 4H ($=, <, <=$)			4H, 2W ($=, \text{not} =, >, <=$)
Shift right/left		8B, 4H	4H		
Shift right arithmetic		4H	4H		
Multiply and add		8B, 4H			
Shift and add (saturating)			4H		
And/or/xor	8B, 4H, 2W	8B, 4H, 2W	8B, 4H, 2W		8B, 4H, 2W
Absolute difference	8B				8B
Max/min	8B, 4W	8B, 4H			
Pack ($2n$ bits $\rightarrow n$ bits)	$2W \rightarrow 2B, 4H \rightarrow 4B$	$2*2W \rightarrow 4H, 2*4H \rightarrow 8B$	$2*4H \rightarrow 8B$		$2W \rightarrow 2H, 2W \rightarrow 2B, 4H \rightarrow 4B$
Unpack/merge	$2B \rightarrow 2W, 4B \rightarrow 4H$	$2*4B \rightarrow 8B, 2*2H \rightarrow 4H$			$4B \rightarrow 4H, 2*4B \rightarrow 8B$
Permute/shuffle		8B, 4H	4H		
Register sets	Integer	Fl. Pt. + 192b Acc.	Integer		Fl. Pt.

FIGURE D.4.1 Summary of multimedia support for desktop RISCs. B stands for byte (8 bits), H for half word (16 bits), and W for word (32 bits). Thus 8B means an operation on eight bytes in a single instruction. Pack and unpack use the notation $2*2W$ to mean two operands each with two words. Note that MDMX has vector/scalar operations, where the scalar is specified as an element of one of the vector registers. This table is a simplification of the full multimedia architectures, leaving out many details. For example, MIPS MDMX includes instructions to multiplex between two operands, HP MAX2 includes an instruction to calculate averages, and SPARC VIS includes instructions to set registers to constants. Also, this table does not include the memory alignment operation of MDMX, MAX, and VIS.

per field. This wide accumulator can be used for add, subtract, and multiply/ add instructions. MIPS claims performance advantages of two to four times for the accumulator.

Perhaps the surprising conclusion of this table is the lack of consistency. The only operations found on all four are the logical operations (AND, OR, XOR), which do not need a partitioned ALU. If we leave out the frugal Alpha, then the only other common operations are parallel adds and subtracts on four halfwords.

Each manufacturer states that these are instructions intended to be used in hand-optimized subroutine libraries, an intention likely to be followed, as a compiler that works well with multimedia extensions of all desktop RISCs would be challenging.

D.5

Instructions: Digital Signal-Processing Extensions of the Embedded RISCs

One feature found in every digital signal processor (DSP) architecture is support for integer multiply-accumulate. The multiplies tend to be on shorter words than regular integers, such as 16 bits, and the accumulator tends to be on longer words, such as 64 bits. The reason for multiply-accumulate is to efficiently implement digital filters, common in DSP applications. Since Thumb and MIPS-16 are subset architectures, they do not provide such support. Instead, programmers should use the DSP or multimedia extensions found in the 32-bit mode instructions of ARM and MIPS-64.

Figure D.5.1 shows the size of the multiply, the size of the accumulator, and the operations and instruction names for the embedded RISCs. Machines with accumulator sizes greater than 32 and less than 64 bits will force the upper bits to remain as the sign bits, thereby “saturating” the add to set to maximum and minimum fixed-point values if the operations overflow.

	ARMv4	Thumb	SuperH	M32R	MIPS-16
Size of multiply	32B × 32B	—	32B × 32B, 16B × 16B	32B × 16B, 16B × 16B	—
Size of accumulator	32B/64B	—	32B/42B, 48B/64B	56B	—
Accumulator name	Any GPR or pairs of GPRs	—	MACH, MACL	ACC	—
Operations	32B/64B product + 64B accumulate signed/unsigned	—	32B product + 42B/32B accumulate (operands in memory); 64B product + 64B/48B accumulate (operands in memory); clear MAC	32B/48B product + 64B accumulate, round, move	—
Corresponding instruction names	MLA, SMLAL, UMLAL	—	MAC, MACS, MAC.L, MAC.LS, CLRMAC	MACHI/MACLO, MACWHI/MACWLO, RAC, RACH, MVFACHI/MVFACLO, MVTACHI/MVTACLO	—

FIGURE D.5.1 Summary of five embedded RISC approaches to multiply-accumulate.

D.6

Instructions: Common Extensions to MIPS Core

Figures D.6.1 through D.6.7 list instructions not found in Figures D.3.5 through D.3.11 in the same four categories. Instructions are put in these lists if they appear in more than one of the standard architectures. The instructions are defined using the hardware description language defined in Figure D.6.8.

Although most of the categories are self-explanatory, a few bear comment:

- The “atomic swap” row means a primitive that can exchange a register with memory without interruption. This is useful for operating system semaphores in a uniprocessor as well as for multiprocessor synchronization (see Section 2.11 in Chapter 2).
- The 64-bit data transfer and operation rows show how MIPS, PowerPC, and SPARC define 64-bit addressing and integer operations. SPARC simply defines all register and addressing operations to be 64 bits, adding only

Name	Definition	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Atomic swap R/M (for locks and semaphores)	Temp $\leftarrow$ Rd; Rd $\leftarrow$ Mem[x]; Mem[x] $\leftarrow$ Temp	LDL/Q_L; STL/Q_C	LL; SC	— (see D.8)	LWARX; STWCX	CASA, CASX
Load 64-bit integer	Rd $\leftarrow$ ₆₄ Mem[x]	LDQ	LD	LDD	LD	LDX
Store 64-bit integer	Mem[x] $\leftarrow$ ₆₄ Rd	STQ	SD	STD	STD	STX
Load 32-bit integer unsigned	Rd _{32..63} $\leftarrow$ ₃₂ Mem[x]; Rd _{0..31} $\leftarrow$ ₃₂ 0	LDL; EXTLL	LWU	LDW	LWZ	LDUW
Load 32-bit integer signed	Rd _{32..63} $\leftarrow$ ₃₂ Mem[x]; 32 Rd _{0..31} $\leftarrow$ ₃₂ Mem[x] ₀	LDL	LW	LDW; EXTRD,S 63, 8	LWA	LDSW
Prefetch	Cache[x] $\leftarrow$ <i>hint</i>	FETCH, FETCH_M*	PREF, PREFX	LDD, r0 LDW, r0	DCBT, DCBTST	PRE-FETCH
Load coprocessor	Coprocessor $\leftarrow$ Mem[x]	—	LWCi	CLDWX, CLDWS	—	—
Store coprocessor	Mem[x] $\leftarrow$ Coprocessor	—	SWCi	CSTWX, CSTWS	—	—
Endian	(Big/little endian?)	Either	Either	Either	Either	Either
Cache flush	(Flush cache block at this address)	ECB	CP0op	FDC, FIC	DCBF	FLUSH
Shared memory synchronization	(All prior data transfers complete before next data transfer may start)	WMB	SYNC	SYNC	SYNC	MEMBAR

FIGURE D.6.1 Data transfer instructions not found in MIPS core but found in two or more of the five desktop architectures. The load linked/store conditional pair of instructions gives Alpha and MIPS atomic operations for semaphores, allowing data to be read from memory, modified, and stored without fear of interrupts or other machines accessing the data in a multiprocessor (see Chapter 2). Prefetching in the Alpha to external caches is accomplished with FETCH and FETCH_M; on-chip cache prefetches use LD_Q A, R31, and LD_Y A. F31 is used in the Alpha 21164 (see Bhandarkar [1995], p. 190).

Name	Definition	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
64-bit integer arithmetic ops	$Rd \leftarrow {}_{64}Rs_1 \text{ op}_{64} Rs_2$	ADD, SUB, MUL	DADD, DSUB, DMULT, DDIV	ADD, SUB, SHLADD, DS	ADD, SUBF, MULLD, DIVD	ADD, SUB, MULX, S/UDIVX
64-bit integer logical ops	$Rd \leftarrow {}_{64}Rs_1 \text{ op}_{64} Rs_2$	AND, OR, XOR	AND, OR, XOR	AND, OR, XOR	AND, OR, XOR	AND, OR, XOR
64-bit shifts	$Rd \leftarrow {}_{64}Rs_1 \text{ op}_{64} Rs_2$	SLL, SRA, SRL	DSLL/V, DSRA/V, DSRL/V	DEPD,Z, EXTRD,S, EXTRD,U	SLD, SRAD, SRLD	SLLX, SRAX, SRLX
Conditional move	if (cond) $Rd \leftarrow Rs$	CMOV_	MOVN/Z	SUBc, n; ADD	—	MOVcc, MOVr
Support for multiword integer add	CarryOut, $Rd \leftarrow Rs_1 + Rs_2 + \text{OldCarryOut}$	—	ADU; SLTU; ADDU, DADU; SLTU; DADDU	ADDC	ADDC, ADDE	ADDcc
Support for multiword integer sub	CarryOut, $Rd \leftarrow Rs_1 - Rs_2 + \text{OldCarryOut}$	—	SUBU; SLTU; SUBU, DSUBU; SLTU; DSUBU	SUBB	SUBFC, SUBFE	SUBcc
And not	$Rd \leftarrow Rs_1 \& \sim(Rs_2)$	BIC	—	ANDCM	ANDC	ANDN
Or not	$Rd \leftarrow Rs_1 \sim(Rs_2)$	ORNOT	—	—	ORC	ORN
Add high immediate	$Rd_{0..15} \leftarrow Rs_{0..15} + (\text{Const} \ll 16)$	—	—	ADDIL (R-I)	ADDIS (R-I)	—
Coprocessor operations	(Defined by coprocessor)	—	COPi	COPR,i	—	IMPDEPi

FIGURE D.6.2 Arithmetic/logical instructions not found in MIPS core but found in two or more of the five desktop architectures.

Name	Definition	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Optimized delayed branches	(Branch not always delayed)	—	BEQL, BNEL, B_ZL (<, >, <=, >=)	COMBT, n, COMBF, n	—	BPcc, A, FPBcc, A
Conditional trap	if (COND) { $R31 \leftarrow PC$; $PC \leftarrow 0..0\#i$ }	—	T_., T_I (=, not=, <, >, <=, >=)	SUBc, n; BREAK	TW, TD, TWI, TDI	Tcc
No. control registers	Misc. regs (virtual memory, interrupts, . . .)	6	equiv. 12	32	33	29

FIGURE D.6.3 Control instructions not found in MIPS core but found in two or more of the five desktop architectures.

special instructions for 64-bit shifts, data transfers, and branches. MIPS includes the same extensions, plus it adds separate 64-bit signed arithmetic instructions. PowerPC adds 64-bit right shift, load, store, divide, and compare and has a separate mode determining whether instructions are interpreted as 32- or 64-bit operations; 64-bit operations will not work in a machine that

Name	Definition	Alpha	MIPS-64	PA-RISC 2.0	PowerPC	SPARCv9
Multiply and add	$Fd \leftarrow (Fs1 \times Fs2) + Fs3$	—	MADD.S/D	FMPYFADD sg1/db1	FMADD/S	
Multiply and sub	$Fd \leftarrow (Fs1 \times Fs2) - Fs3$	—	MSUB.S/D		FMSUB/S	
Neg mult and add	$Fd \leftarrow -((Fs1 \times Fs2) + Fs3)$	—	NMADD.S/D	FMPYFNEG sg1/db1	FNMADD/S	
Neg mult and sub	$Fd \leftarrow -((Fs1 \times Fs2) - Fs3)$	—	NMSUB.S/D		FNMSUB/S	
Square root	$Fd \leftarrow \text{SQRT}(Fs)$	SQRT_	SQRT.S/D	FSQRT sg1/db1	FSQRT/S	FSQRTS/D
Conditional move	if (cond) $Fd \leftarrow Fs$	FCMOV_	MOVF/T, MOVF/T.S/D	FTESTFCPY	—	FMOVcc
Negate	$Fd \leftarrow Fs \wedge x80000000$	CPYSN	NEG.S/D	FNEG sg1/db1	FNEG	FNEGS/D/Q
Absolute value	$Fd \leftarrow Fs \& x7FFFFFFF$	—	ABS.S/D	FABS/db1	FABS	FABSS/D/Q

FIGURE D.6.4 Floating-point instructions not found in MIPS core but found in two or more of the five desktop architectures.

Name	Definition	ARMv4	Thumb	SuperH	M32R	MIPS-16
Atomic swap R/M (for semaphores)	Temp $\leftarrow$ Rd; Rd $\leftarrow$ Mem[x]; Mem[x] $\leftarrow$ Temp	SWP, SWPB	— ¹	(see TAS)	LOCK; UNLOCK	— ¹
Memory management unit	Paged address translation	Via coprocessor instructions	— ¹	LDTLB		— ¹
Endian	(Big/little endian?)	Either	Either	Either	Big	Either

FIGURE D.6.5 Data transfer instructions not found in MIPS core but found in two or more of the five embedded architectures. We use —¹ to show sequences that are available in 32-bit mode but not 16-bit mode in Thumb or MIPS-16.

only supports 32-bit mode. PA-RISC is expanded to 64-bit addressing and operations in version 2.0.

- The “prefetch” instruction supplies an address and hint to the implementation about the data. Hints include whether the data are likely to be read or written soon, likely to be read or written only once, or likely to be read or written many times. Prefetch does not cause exceptions. MIPS has a version that adds two registers to get the address for floating-point programs, unlike nonfloating-point MIPS programs.
- In the “Endian” row, “Big/little” means there is a bit in the program status register that allows the processor to act either as big endian or little endian (see [Appendix A](#)). This can be accomplished by simply complementing some of the least significant bits of the address in data transfer instructions.

- The “shared memory synchronization” helps with cache-coherent multi-processors: all loads and stores executed before the instruction must complete before loads and stores after it can start. (See [Chapter 2](#).)
- The “coprocessor operations” row lists several categories that allow for the processor to be extended with special-purpose hardware.

Name	Definition	ARMv4	Thumb	SuperH	M32R	MIPS-16
Load immediate	$Rd \leftarrow Imm$	MOV	MOV	MOV, MOVA	LDI, LD24	LI
Support for multiword integer add	$CarryOut, Rd \leftarrow Rd + Rs1 + OldCarryOut$	ADCS	ADC	ADDC	ADDX	— ¹
Support for multiword integer sub	$CarryOut, Rd \leftarrow Rd - Rs1 + OldCarryOut$	SBCS	SBC	SUBC	SUBX	— ¹
Negate	$Rd \leftarrow 0 - Rs1$		NEG ²	NEG	NEG	NEG
Not	$Rd \leftarrow \sim(Rs1)$	MVN	MVN	NOT	NOT	NOT
Move	$Rd \leftarrow Rs1$	MOV	MOV	MOV	MV	MOVE
Rotate right	$Rd \leftarrow Rs\ i, \gg\ Rd_{0..i-1} \leftarrow Rs_{31-i..31}$	ROR	ROR	ROTC		
And not	$Rd \leftarrow Rs1 \& \sim(Rs2)$	BIC	BIC			

FIGURE D.6.6 Arithmetic/logical instructions not found in MIPS core but found in two or more of the five embedded architectures. We use —¹ to show sequences that are available in 32-bit mode but not in 16-bit mode in Thumb or MIPS-16. The superscript 2 shows new instructions found only in 16-bit mode of Thumb or MIPS-16, such as NEG².

Name	Definition	ARMv4	Thumb	SuperH	M32R	MIPS-16
No. control registers	Misc. registers	21	29	9	5	36

FIGURE D.6.7 Control information in the five embedded architectures.

One difference that needs a longer explanation is the optimized branches. [Figure D.6.9](#) shows the options. The Alpha and PowerPC offer branches that take effect immediately, like branches on earlier architectures. To accelerate branches, these machines use branch prediction (see [Chapter 4](#)). All the rest of the desktop RISCs offer delayed branches. The embedded RISCs generally do not support delayed branch, with the exception of SuperH, which has it as an option.

The other three desktop RISCs provide a version of delayed branch that makes it easier to fill the delay slot. The SPARC “annulling” branch executes the instruction in the delay slot only if the branch is taken; otherwise the instruction is annulled. This means the instruction at the target of the branch can safely be copied into the delay slot, since it will only be executed if the branch is taken. The restrictions are that the target is not another branch and that the target is known at compile time. (SPARC also offers a nondelayed jump because an unconditional branch with the annul bit set does not execute the following instruction.) Later versions of the MIPS

Notation	Meaning	Example	Meaning
<--	Data transfer. Length of transfer is given by the destination's length; the length is specified when not clear.	Regs[R1]<--Regs[R2];	Transfer contents of R2 to R1. Registers have a fixed length, so transfers shorter than the register size must indicate which bits are used.
M	Array of memory accessed in bytes. The starting address for a transfer is indicated as the index to the memory array.	Regs[R1]<--M[x];	Place contents of memory location x into R1. If a transfer starts at M[i] and requires 4 bytes, the transferred bytes are M[i], M[i+1], M[i+2], and M[i+3].
<--n	Transfer an <i>n</i> -bit field, used whenever length of transfer is not clear.	M[y]<-- ₁₆ M[x];	Transfer 16 bits starting at memory location x to memory location y. The length of the two sides should match.
X _n	Subscript selects a bit.	Regs[R1] ₀ <--0;	Change sign bit of R1 to 0. (Bits are numbered from MSB starting at 0.)
X _{m..n}	Subscript selects a field.	Regs[R3] _{24..31} <--M[x];	Moves contents of memory location x into low-order byte of R3.
X ⁿ	Superscript replicates a bit field.	Regs[R3] _{0..23} <--024;	Sets high-order three bytes of R3 to 0.
##	Concatenates two fields.	Regs[R3]<-- ²⁴ 0##M[x]; F2##F3<-- ₆₄ M[x];	Moves contents of location x into low byte of R3; clears upper three bytes. Moves 64 bits from memory starting at location x; 1st 32 bits go into F2, 2nd 32 into F3.
, &	Dereference a pointer; get the address of a variable.	p<--&x;	Assign to object pointed to by p the address of the variable x.
<<, >>	C logical shifts (left, right).	Regs[R1]<<5	Shift R1 left 5 bits.
==, !=, >, <, >=, <=	C relational operators; equal, not equal, greater, less, greater or equal, less or equal.	(Regs[R1]==Regs[R2]) & (Regs[R3]!=Regs[R4])	True if contents of R1 equal the contents of R2 and contents of R3 do not equal the contents of R4.
&, , ^, !	C bitwise logical operations: AND, OR, exclusive OR, and complement.	(Regs[R1] & (Regs[R2] Regs[R3]))	Bitwise AND of R1 and bitwise OR of R2 and R3.

FIGURE D.6.8 Hardware description notation (and some standard C operators).

	(Plain) branch	Delayed branch	Annulling delayed branch	
Found in architectures	Alpha, PowerPC, ARM, Thumb, SuperH, M32R, MIPS-16	MIPS-64, PA-RISC, SPARC, SuperH	MIPS-64, SPARC	PA-RISC
Execute following instruction	Only if branch <i>not</i> taken	Always	Only if branch taken	If forward branch <i>not</i> taken or backward branch taken

FIGURE D.6.9 When the instruction following the branch is executed for three types of branches.

architecture have added a branch likely instruction that also annuls the following instruction if the branch is not taken. PA-RISC allows almost any instruction to annul the next instruction, including branches. Its “nullifying” branch option will execute the next instruction depending on the direction of the branch and whether it is taken (i.e., if a forward branch is not taken or a backward branch is taken). Presumably this choice was made to optimize loops, allowing the instructions following the exit branch and the looping branch to execute in the common case.

Now that we have covered the similarities, we will focus on the unique features of each architecture. We first cover the desktop/server RISCs, ordering them by length of description of the unique features from shortest to longest, and then the embedded RISCs.

D.7

Instructions Unique to MIPS-64

MIPS has gone through five generations of instruction sets, and this evolution has generally added features found in other architectures. Here are the salient unique features of MIPS, the first several of which were found in the original instruction set.

Nonaligned Data Transfers

MIPS has special instructions to handle misaligned words in memory. A rare event in most programs, it is included for supporting 16-bit minicomputer applications and for doing `memcpy` and `strcpy` faster. Although most RISCs trap if you try to load a word or store a word to a misaligned address, on all architectures misaligned words can be accessed without traps by using four load byte instructions and then assembling the result using shifts and logical ORs. The MIPS load and store word left and right instructions (`LWL`, `LWR`, `SWL`, `SWR`) allow this to be done in just two instructions: `LWL` loads the left portion of the register and `LWR` loads the right portion of the register. `SWL` and `SWR` do the corresponding stores. [Figure D.7.1](#) shows how they work. There are also 64-bit versions of these instructions.

Remaining Instructions

Below is a list of the remaining unique details of the MIPS-64 architecture:

- **NOR**—This logical instruction calculates $\sim(\text{Rs1} \mid \text{Rs2})$.
- **Constant shift amount**—Nonvariable shifts use the 5-bit constant field shown in the register-register format in [Figure D.2.3](#).
- **SYSCALL**—This special trap instruction is used to invoke the operating system.

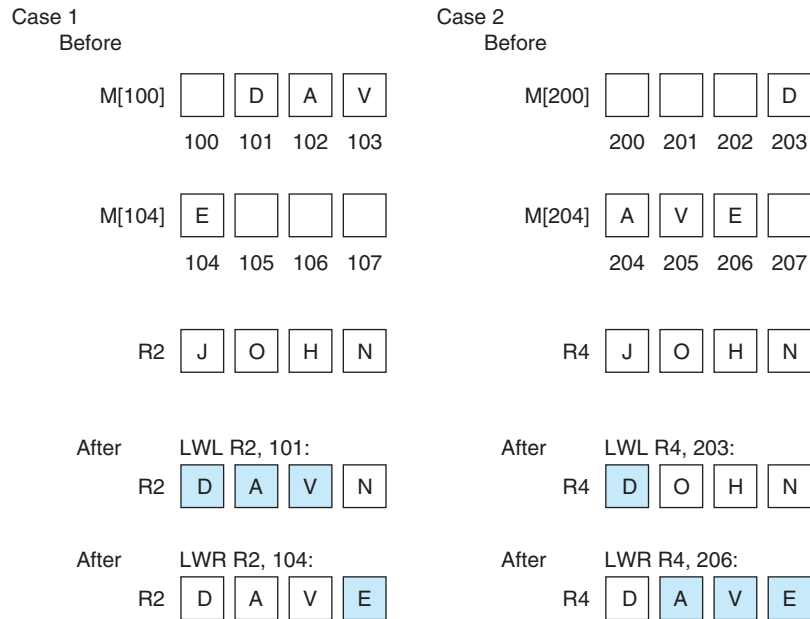


FIGURE D.7.1 MIPS instructions for unaligned word reads. This figure assumes operation in big-endian mode. Case 1 first loads the three bytes 101, 102, and 103 into the left of R2, leaving the least significant byte undisturbed. The following LWR simply loads byte 104 into the least significant byte of R2, leaving the other bytes of the register unchanged using LWL. Case 2 first loads byte 203 into the most significant byte of R4, and the following LWR loads the other three bytes of R4 from memory bytes 204, 205, and 206. LWL reads the word with the first byte from memory, shifts to the left to discard the unneeded byte(s), and changes only those bytes in Rd. The byte(s) transferred are from the first byte to the lowest-order byte of the word. The following LWR addresses the last byte, right-shifts to discard the unneeded byte(s), and finally changes only those bytes of Rd. The byte(s) transferred are from the last byte up to the highest-order byte of the word. Store word left (SWL) is simply the inverse of LWL, and store word right (SWR) is the inverse of LWR. Changing to little-endian mode flips which bytes are selected and discarded. (If big-little, left-right, load-store seem confusing, don't worry; they work!)

- *Move to/from control registers*—CTCi and CFCi move between the integer registers and control registers.
- *Jump/call not PC-relative*—The 26-bit address of jumps and calls is not added to the PC. It is shifted left two bits and replaces the lower 28 bits of the PC. This would only make a difference if the program were located near a 256 MB boundary.
- *TLB instructions*—*Translation-lookaside buffer* (TLB) misses were handled in software in MIPS I, so the instruction set also had instructions for manipulating the registers of the TLB (see [Chapter 5](#) for more on TLBs). These registers are considered part of the “system coprocessor.” Since MIPS I

the instructions differ among versions of the architecture; they are more part of the implementations than part of the instruction set architecture.

- *Reciprocal and reciprocal square root*—These instructions, which do not follow IEEE 754 guidelines of proper rounding, are included apparently for applications that value speed of divide and square root more than they value accuracy.
- *Conditional procedure call instructions*—BGEZAL saves the return address and branches if the content of Rs1 is greater than or equal to zero, and BLTZAL does the same for less than zero. The purpose of these instructions is to get a PC-relative call. (There are “likely” versions of these instructions as well.)
- *Parallel single-precision floating-point operations*—As well as extending the architecture with parallel integer operations in MDMX, MIPS-64 also supports two parallel 32-bit floating-point operations on 64-bit registers in a single instruction. “Paired single” operations include add (ADD.PS), subtract (SUB.PS), compare (C.____.PS), convert (CVT.PS.S, CVT.S.PL, CVT.S.PU), negate (NEG.PS), absolute value (ABS.PS), move (MOV.PS, MOVF.PS, MOVT.PS), multiply (MUL.PS), multiply-add (MADD.PS), and multiply-subtract (MSUB.PS).

There is no specific provision in the MIPS architecture for floating-point execution to proceed in parallel with integer execution, but the MIPS implementations of floating point allow this to happen by checking to see if arithmetic interrupts are possible early in the cycle. Normally, exception detection would force serialization of execution of integer and floating-point operations.

D.8 Instructions Unique to Alpha

The Alpha was intended to be an architecture that made it easy to build high-performance implementations. Toward that goal, the architects originally made two controversial decisions: imprecise floating-point exceptions and no byte or halfword data transfers.

To simplify pipelined execution, Alpha does not require that an exception should act as if no instructions past a certain point are executed and that all before that point have been executed. It supplies the TRAPB instruction, which stalls until all prior arithmetic instructions are guaranteed to complete without incurring arithmetic exceptions. In the most conservative mode, placing one TRAPB per exception-causing instruction slows execution by roughly five times but provides precise exceptions (see [Darcy and Gay \[1996\]](#)).

Code that does not include `TRAPB` does not obey the IEEE 754 floating-point standard. The reason is that parts of the standard (NaNs, infinities, and denormals) are implemented in software on Alpha, as they are on many other microprocessors. To implement these operations in software, however, programs must find the offending instruction and operand values, which cannot be done with imprecise interrupts!

When the architecture was developed, it was believed by the architects that byte loads and stores would slow down data transfers. Byte loads require an extra shifter in the data transfer path, and byte stores require that the memory system perform a read-modify-write for memory systems with error correction codes, since the new ECC value must be recalculated. This omission meant that byte stores required the sequence load word, replaced the desired byte, and then stored the word. (Inconsistently, floating-point loads go through considerable byte swapping to convert the obtuse VAX floating-point formats into a canonical form.)

To reduce the number of instructions to get the desired data, Alpha includes an elaborate set of byte manipulation instructions: extract field and zero rest of a register (`EXTxx`), insert field (`INSxx`), mask rest of a register (`MSKxx`), zero fields of a register (`ZAP`), and compare multiple bytes (`CMPGE`).

Apparently, the implementors were not as bothered by load and store byte as were the original architects. Beginning with the shrink of the second version of the Alpha chip (21164A), the architecture does include loads and stores for bytes and halfwords.

Remaining Instructions

Below is a list of the remaining unique instructions of the Alpha architecture:

- *PAL code*—To provide the operations that the VAX performed in microcode, Alpha provides a mode that runs with all privileges enabled, interrupts disabled, and virtual memory mapping turned off for instructions. PAL (privileged architecture library) code is used for TLB management, atomic memory operations, and some operating system primitives. PAL code is called via the `CALL_PAL` instruction.
- *No divide*—Integer divide is not supported in hardware.
- *“Unaligned” load-store*—`LDQ_U` and `STQ_U` load and store 64-bit data using addresses that ignore the least significant three bits. Extract instructions then select the desired unaligned word using the lower address bits. These instructions are similar to `LWL/R`, `SWL/R` in MIPS.
- *Floating-point single precision represented as double precision*—Single-precision data are kept as conventional 32-bit formats in memory but are converted to 64-bit double-precision format in registers.
- *Floating-point register F31 is fixed at zero*—To simplify comparisons to zero.